

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

“JnanaSangama”, Belgaum -590014, Karnataka.



LAB REPORT
on

OPERATING SYSTEMS

Submitted by

Naysha Navin Shrivastava (1WA24CS184)

in partial fulfillment for the award of the degree of
BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING



B.M.S. COLLEGE OF ENGINEERING

(Autonomous Institution under VTU)

BENGALURU-560019

Feb-2026 to June-2026

B. M. S. College of Engineering,
Bull Temple Road, Bangalore 560019

(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering



CERTIFICATE

This is to certify that the Lab work entitled “**OPERATING SYSTEMS – 23CS4PCOPS**” carried out by **NAYSHA NAVIN SHRIVASTAVA (1WA24CS184)**, who is a Bonafide student of B. M. S. College of Engineering. It is in partial fulfilment for the award of Bachelor of Engineering in Computer Science and Engineering of the Visvesvaraya Technological University, Belgaum during the year **Feb-2026 to June-2026**.

The Lab report has been approved as it satisfies the academic requirements in respect of a **OPERATING SYSTEMS - (23CS4PCOPS)** work prescribed for the said degree.

Dr. Seema Patil
Assistant Professor
Department of CSE
BMSCE, Bengaluru

Dr. Kavitha Sooda
Professor and Head
Department of CSE
BMSCE, Bengaluru

Index Sheet

Sl. No.	Experiment Title	Page No.
1.	Write a C program to simulate the following CPU scheduling algorithm to find turnaround time and waiting time. a) FCFS b) SJF (Preemptive and Non-Preemptive) c) Priority (Preemptive and Non-Preemptive) d) Round Robin (Experiment with different quantum sizes for RR algorithm)	5-23
2.	Write a C program to simulate multi-level queue scheduling algorithm considering the following scenario. All the processes in the system are divided into two categories – system processes and user processes. System processes are to be given higher priority than user processes. Use FCFS scheduling for the processes in each queue.	23-27
3.	Write a C program to simulate Real-Time CPU Scheduling algorithms: a) Rate- Monotonic b) Earliest-deadline First c) Proportional scheduling	28-36
4.	Write a C program to simulate: a) Producer-Consumer problem using semaphores. b) Dining-Philosopher's problem	37-42
5.	Write a C program to simulate: a) Bankers' algorithm for the purpose of deadlock avoidance. b) Deadlock Detection	42-49
6.	Write a C program to simulate the following contiguous memory allocation techniques. a) Worst-fit b) Best-fit c) First-fit	50-53
7.	Write a C program to simulate page replacement algorithms. a) FIFO b) LRU c) Optimal	54-59

Course Outcomes:

C01	Apply the different concepts and functionalities of Operating System
C02	Analyse various Operating system strategies and techniques
C03	Demonstrate the different functionalities of Operating System.
C04	Conduct practical experiments to implement the functionalities of Operating system.

1. Write a C program to simulate the following CPU scheduling algorithm to find turnaround time and waiting time.

a) FCFS

b) SJF (Preemptive and Non-Preemptive)

c) Priority (Preemptive and Non-Preemptive)

d) Round Robin (Experiment with different quantum sizes for RR algorithm)

1.FCFS

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int n, i;
```

```
    printf("Enter total number of processes: ");
```

```
    scanf("%d", &n);
```

```
    int at[n], bt[n], ct[n], tat[n], wt[n];
```

```
    // Input
```

```
    for(i = 0; i < n; i++)
```

```
    {
```

```
        printf("\nProcess [%d]\n", i+1);
```

```
        printf("Arrival time: ");
```

```
        scanf("%d", &at[i]);
```

```
        printf("Burst time: ");
```

```
        scanf("%d", &bt[i]);
```

```
    }
```

```
    int current_time = 0;
```

```

// FCFS Calculation
for(i = 0; i < n; i++)
{
    if(current_time < at[i])
        current_time = at[i];

    ct[i] = current_time + bt[i];
    tat[i] = ct[i] - at[i];
    wt[i] = tat[i] - bt[i];

    current_time = ct[i];
}

float avg_tat = 0, avg_wt = 0;

// Output Table
printf("\nProcess Arrival Time Burst Time Completion Time Turnaround Time Waiting Time\n");

for(i = 0; i < n; i++)
{
    printf("%d\t%d\t%d\t%d\t%d\t%d\n",
        i+1, at[i], bt[i], ct[i], tat[i], wt[i]);

    avg_tat += tat[i];
    avg_wt += wt[i];
}

avg_tat /= n;
avg_wt /= n;

```

```

printf("\nAverage Turnaround Time: %.2f", avg_tat);
printf("\nAverage Waiting Time: %.2f\n", avg_wt);

return 0;
}

```

```

Enter total number of processes: 3

Process [1]
Arrival time: 0
Burst time: 6

Process [2]
Arrival time: 3
Burst time: 8

Process [3]
Arrival time: 7
Burst time: 2

Process Arrival Time Burst Time Completion Time Turnaround Time Waiting Time
1      0      6      6      6      0
2      3      8     14     11      3
3      7      2     16      9      7

Average Turnaround Time: 8.67
Average Waiting Time: 3.33

Process returned 0 (0x0)   execution time : 28.683 s
Press any key to continue.
|

```

2.SJF

(A)Non Pre-emptive

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int n, i, j;
```

```
    printf("USN: 1WA24CS198");
```

```
    printf("\nEnter number of processes: ");
```

```
    scanf("%d", &n);
```

```
    int at[n], bt[n], ct[n], tat[n], wt[n];
```

```
    int finished[n];
```

```
    for(i = 0; i < n; i++)
```

```
        finished[i] = 0;
```

```
    printf("Enter arrival times:\n");
```

```
    for(i = 0; i < n; i++)
```

```
        scanf("%d", &at[i]);
```

```
    printf("Enter burst times:\n");
```

```
    for(i = 0; i < n; i++)
```

```
        scanf("%d", &bt[i]);
```

```
    int current_time = 0, completed = 0;
```



```

int min_index;

while(completed < n)
{
    min_index = -1;
    int min_bt = 10000;

    for(i = 0; i < n; i++)
    {
        if(at[i] <= current_time && finished[i] == 0)
        {
            if(bt[i] < min_bt)
            {
                min_bt = bt[i];
                min_index = i;
            }
        }
    }

    if(min_index == -1)
    {
        current_time++;
    }
    else
    {
        ct[min_index] = current_time + bt[min_index];
        tat[min_index] = ct[min_index] - at[min_index];
        wt[min_index] = tat[min_index] - bt[min_index];

        current_time = ct[min_index];
        finished[min_index] = 1;
    }
}

```

```
        completed++;
    }
}

float avg_tat = 0, avg_wt = 0;

printf("\nSJF scheduling:\n");
printf("PID\tAT\tBT\tCT\tTAT\tWT\n");

for(i = 0; i < n; i++)
{
    printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
        i+1, at[i], bt[i], ct[i], tat[i], wt[i]);

    avg_tat += tat[i];
    avg_wt += wt[i];
}

avg_tat /= n;
avg_wt /= n;

printf("\nAverage turnaround time:%fms\n", avg_tat);
printf("\nAverage waiting time:%fms\n", avg_wt);

return 0;
}
```

```
C:\Users\BMSCECSE-14\Desktop X + v
USN: 1WA24CS184
Enter number of processes: 5
Enter arrival times:
0
3
6
8
4
Enter burst times:
6
4
9
2
1

SJF scheduling:
PID    AT    BT    CT    TAT    WT
P1      0     6     6     6     0
P2      3     4    11     8     4
P3      6     9    22    16     7
P4      8     2    13     5     3
P5      4     1     7     3     2

Average turnaround time:7.600000ms

Average waiting time:3.200000ms

Process returned 0 (0x0)   execution time : 22.610 s
Press any key to continue.
```

3.SRTF

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int n, i, time = 0, completed = 0, min_index;
```

```
    printf("USN: 1WA24CS198");
```

```
    printf("\nEnter the number of processes: ");
```

```
    scanf("%d", &n);
```

```
    int at[n], bt[n], rt[n], ct[n], tat[n], wt[n];
```

```
    for(i = 0; i < n; i++)
```

```
    {
```

```
        printf("Enter arrival time and burst time for process %d: ", i+1);
```

```
        scanf("%d %d", &at[i], &bt[i]);
```

```
        rt[i] = bt[i]; // remaining time
```

```
    }
```

```
    while(completed < n)
```

```
    {
```

```
        min_index = -1;
```

```
        int min_rt = 10000;
```

```
        for(i = 0; i < n; i++)
```

```
        {
```

```
            if(at[i] <= time && rt[i] > 0)
```

```
            {
```

```
                if(rt[i] < min_rt)
```

```
                {
```

```

        min_rt = rt[i];
        min_index = i;
    }
}

if(min_index == -1)
{
    time++;
}
else
{
    rt[min_index]--;
    time++;

    if(rt[min_index] == 0)
    {
        completed++;
        ct[min_index] = time;
        tat[min_index] = ct[min_index] - at[min_index];
        wt[min_index] = tat[min_index] - bt[min_index];
    }
}

float avg_wt = 0, avg_tat = 0;

printf("\nPID\tAT\tBT\tCT\tTAT\tWT\n");

for(i = 0; i < n; i++)
{

```

```

printf("%d\t%d\t%d\t%d\t%d\t%d\n",
i+1, at[i], bt[i], ct[i], tat[i], wt[i]);

avg_wt += wt[i];
avg_tat += tat[i];
}

avg_wt /= n;
avg_tat /= n;

printf("\nAverage Waiting Time: %.2f\n", avg_wt);
printf("Average Turnaround Time: %.2f\n", avg_tat);

return 0;
}

```

```

C:\Users\BMSCECSE-14\Desktop
USN: 1WA24CS184
Enter the number of processes: 5
Enter arrival time and burst time for process 1: 5
9
Enter arrival time and burst time for process 2: 8
6
Enter arrival time and burst time for process 3: 3
1
Enter arrival time and burst time for process 4: 0
5
Enter arrival time and burst time for process 5: 2
4

PID    AT    BT    CT    TAT    WT
1       5     9    25    20     11
2       8     6    16     8      2
3       3     1     4     1      0
4       0     5     6     6      1
5       2     4    10     8      4

Average Waiting Time: 3.60
Average Turnaround Time: 8.60

Process returned 0 (0x0)   execution time : 32.728 s
Press any key to continue.

```

Priority scheduling

(A) non-pre-emptive

```
#include <stdio.h>

int main()
{
    int n;
    printf("Naysha Navin Shrivastava");
    printf("1WA24CS184");

    printf("Enter number of processes: ");
    scanf("%d",&n);

    int pid[n], at[n], bt[n], pr[n];
    int ct[n], tat[n], wt[n];
    int completed[n];

    for(int i=0;i<n;i++)
        completed[i] = 0;

    // Input process details
    for(int i=0;i<n;i++)
    {
        printf("\nEnter details for Process %d\n",i+1);

        pid[i] = i+1;

        printf("Arrival Time: ");
        scanf("%d",&at[i]);

        printf("Burst Time: ");
        scanf("%d",&bt[i]);

        printf("Priority (Lower number = Higher priority): ");
        scanf("%d",&pr[i]);
    }

    int current_time = 0;
    int completed_count = 0;

    while(completed_count < n)
    {
        int highest_priority = 9999;
```

```

int selected = -1;

for(int i=0;i<n;i++)
{
    if(at[i] <= current_time && completed[i] == 0)
    {
        if(pr[i] < highest_priority)
        {
            highest_priority = pr[i];
            selected = i;
        }
    }
}

if(selected == -1)
{
    current_time++;
}
else
{
    ct[selected] = current_time + bt[selected];

    tat[selected] = ct[selected] - at[selected];

    wt[selected] = tat[selected] - bt[selected];

    current_time = ct[selected];

    completed[selected] = 1;
    completed_count++;
}
}

printf("\nPID\tAT\tBT\tPR\tCT\tTAT\tWT\n");

for(int i=0;i<n;i++)
{
    printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
        pid[i],at[i],bt[i],pr[i],ct[i],tat[i],wt[i]);
}

return 0;
}

```



```

C:\Users\Admin\Desktop\1W X + v
Naysha Navin Shrivastava\WA24CS184)Enter number of processes: 5

Enter details for Process 1
Arrival Time: 0
Burst Time: 3
Priority (Lower number = Higher priority): 5

Enter details for Process 2
Arrival Time: 2
Burst Time: 2
Priority (Lower number = Higher priority): 3

Enter details for Process 3
Arrival Time: 3
Burst Time: 5
Priority (Lower number = Higher priority): 2

Enter details for Process 4
Arrival Time: 4
Burst Time: 4
Priority (Lower number = Higher priority): 4

Enter details for Process 5
Arrival Time: 6
Burst Time: 1
Priority (Lower number = Higher priority): 1

PID    AT    BT    PR    CT    TAT    WT
P1      0      3      5      3      3      0
P2      2      2      3     11      9      7
P3      3      5      2      8      5      0
P4      4      4      4     15     11      7
P5      6      1      1      9      3      2

Process returned 0 (0x0)  execution time : 99.041 s
Press any key to continue.
```

(B)non-pre-emptive:

```
#include <stdio.h>

int main()
{
    int n, i;

    printf("Enter number of processes: ");
    scanf("%d", &n);
    printf("Nayssha Navin Shrivastava");
    printf("1WA24CS184");

    int pid[n], at[n], bt[n], pr[n];
    int rt[n], ct[n], tat[n], wt[n];

    // Input
    for(i = 0; i < n; i++)
    {
        printf("\nProcess %d\n", i + 1);

        printf("Process ID: ");
        scanf("%d", &pid[i]);

        printf("Arrival Time: ");
        scanf("%d", &at[i]);

        printf("Burst Time: ");
        scanf("%d", &bt[i]);

        printf("Priority: ");
        scanf("%d", &pr[i]);

        rt[i] = bt[i]; // Remaining time
    }

    int current_time = 0;
    int completed = 0;

    while(completed < n)
    {
        int highest_priority = 9999;
```

```

int selected_process = -1;

// Find highest priority process
for(i = 0; i < n; i++)
{
    if(at[i] <= current_time && rt[i] > 0)
    {
        if(pr[i] < highest_priority)
        {
            highest_priority = pr[i];
            selected_process = i;
        }
    }
}

// No process available
if(selected_process == -1)
{
    current_time++;
}
else
{
    rt[selected_process]--;
    current_time++;

    if(rt[selected_process] == 0)
    {
        ct[selected_process] = current_time;

        tat[selected_process] =
            ct[selected_process] - at[selected_process];

        wt[selected_process] =
            tat[selected_process] - bt[selected_process];

        completed++;
    }
}

printf("\nPID\tAT\tBT\tPR\tCT\tTAT\tWT\n");

for(i = 0; i < n; i++)
{

```

```
        printf("%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
               pid[i], at[i], bt[i], pr[i],
               ct[i], tat[i], wt[i]);
    }

    return 0;
}
```

Enter number of processes: 7
Naysha Navin Shrivastava1WA24CS184
Process 1
Process ID: 6
Arrival Time: 6
Burst Time: 5
Priority: 6

Process 2
Process ID: 4
Arrival Time: 4
Burst Time: 1
Priority: 5

Process 3
Process ID: 2
Arrival Time: 1
Burst Time: 2
Priority: 4

Process 4
Process ID: 3
Arrival Time: 3
Burst Time: 4
Priority: 4

Process 5
Process ID: 1
Arrival Time: 0
Burst Time: 8
Priority: 3

Process 6
Process ID: 5
Arrival Time: 5
Burst Time: 6
Priority: 2

Process 7
Process ID: 7
Arrival Time: 10
Burst Time: 1
Priority: 1

PID	AT	BT	PR	CT	TAT	WT
6	6	5	6	27	21	16
4	4	1	5	22	18	17
2	1	2	4	17	16	14
3	3	4	4	21	18	14
1	0	8	3	15	15	7
5	5	6	2	12	7	1

Round Robin

```
#include <stdio.h>
```

```
int main() {
```

```
    int n, tq;
```

```
    int at[20], bt[20], rt[20];
```

```
    int wt[20], tat[20], rt_time[20];
```

```
    int time = 0, completed = 0;
```

```
    float avg_wt = 0, avg_tat = 0, avg_rt = 0;
```

```
    printf("Naysha Navin Shrivastava");
```

```
    printf("1WA24CS184");
```

```
    printf("Enter number of processes: ");
```

```
    scanf("%d",&n);
```

```
    for(int i=0;i<n;i++){
```

```
        printf("Enter Arrival Time and Burst Time for P%d: ",i+1);
```

```
        scanf("%d %d",&at[i],&bt[i]);
```

```
        rt[i] = bt[i];
```

```
        rt_time[i] = -1;
```

```
    }
```

```
    printf("Enter Time Quantum: ");
```

```
    scanf("%d",&tq);
```

```
    while(completed < n){
```

```
        int executed = 0;
```

```
        for(int i=0;i<n;i++){
```

```
            if(at[i] <= time && rt[i] > 0){
```

```
                if(rt_time[i] == -1)
```

```
                    rt_time[i] = time - at[i];
```

```

        if(rt[i] > tq){
            time += tq;
            rt[i] -= tq;
        }
        else{
            time += rt[i];
            rt[i] = 0;

            tat[i] = time - at[i];
            wt[i] = tat[i] - bt[i];

            avg_wt += wt[i];
            avg_tat += tat[i];
            avg_rt += rt_time[i];

            completed++;
        }

        executed = 1;
    }
}

if(executed == 0)
    time++;
}

printf("\nPID\tAT\tBT\tWT\tTAT\tRT\n");

for(int i=0;i<n;i++){
    printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
        i+1,at[i],bt[i],wt[i],tat[i],rt_time[i]);
}

printf("\nAverage Waiting Time = %.2f",avg_wt/n);
printf("\nAverage Turnaround Time = %.2f",avg_tat/n);
printf("\nAverage Response Time = %.2f\n",avg_rt/n);

return 0;
}

```

```
"C:\Users\Admin\Desktop\1W X + v
Naysha Navin Shrivastava1WA24CS184Enter number of processes:
5
Enter Arrival Time and Burst Time for P1: 0
5
Enter Arrival Time and Burst Time for P2: 1
3
Enter Arrival Time and Burst Time for P3: 2
1
Enter Arrival Time and Burst Time for P4: 3
2
Enter Arrival Time and Burst Time for P5: 4
3
Enter Time Quantum: 2

PID    AT    BT    WT    TAT    RT
P1      0     5     9     14     0
P2      1     3     8     11     1
P3      2     1     2     3      2
P4      3     2     2     4      2
P5      4     3     6     9      3

Average Waiting Time = 5.40
Average Turnaround Time = 8.20
Average Response Time = 1.60

Process returned 0 (0x0)  execution time : 35.853 s
Press any key to continue.
```


2. Write a C program to simulate multi-level queue scheduling algorithm considering the following scenario. All the processes in the system are divided into two categories – system processes and user processes. System processes are to be given higher priority than user processes. Use FCFS scheduling for the processes in each queue

```
#include<stdio.h>

#define MAX 20

struct Process (
    int pid, at, bt, ct, tat, wt, type;

int main() (
    struct Process p[MAX], sq[MAX], uq[MAX]; int
    n, i, s    count = 0, u    count = 0, time = 0;
    printf("Enter number of processes: "); scanf("%d",
    &n);
    for (i = 0; i < n; i++) (
        printf("\nProcess %d\n", i + 1);
        p[i].pid = i + 1;
        printf("Arrival Time: ");
        scanf("%d", &p[i].at);
        printf("Burst Time: ");
        scanf("%d", &p[i].bt);
        printf("Type (1=System,2=User): ");
        scanf("%d", &p[i].type);
        if(p[i].type == 1)
            sq[s    count++] = p[i];
        else
            uq[u    count++] = p[i];

    for (i = 0; i < s    count; i++) {
        if(time < sq[i].at)
            time = sq[i].at; time
            += sq[i].bt;
        sq[i].ct = time;

    for (i = 0; i < u    count; i++) (
        if(time < uq[i].at)
            time = uq[i].at; time
            += uq[i].bt;
        uq[i].ct = time;

    float total tat = 0, total wt = 0;
```

```

printf("\n---System Queue---\n");
printf("PID\tAT\tBT\tCT\tTAT\tWT\n");

    for (i = 0; i < s_count; i++) (
        sq[i].tat = sq[i].ct - sq[i].at;
        sq[i].wt = sq[i].tat - sq[i].bt;

total_tat += sq[i].tat;
total_wt += sq[i].wt;

        printf("%d\t%d\t%d\t%d\t%d\t%d\n",
            sq[i].pid, sq[i].at, sq[i].bt,
            sq[i].ct, sq[i].tat, sq[i].wt);

printf("\n---User Queue---\n");
printf("PID\tAT\tBT\tCT\tTAT\tWT\n");

    for (i = 0; i < u_count; i++) {
        uq[i].tat = uq[i].ct - uq[i].at;
        uq[i].wt = uq[i].tat - uq[i].bt;

total_tat += uq[i].tat; total_wt
+= uq[i].wt;

        printf("%d\t%d\t%d\t%d\t%d\t%d\n",
            uq[i].pid, uq[i].at, uq[i].bt,
            uq[i].ct, uq[i].tat, uq[i].wt);

printf("\nAverage Turnaround Time=%.2f", total_tat / n);
printf("\nAverage Waiting Time=%.2f\n", total_wt / n);

printf("Naysha Navin Shrivastav\n"); return 0;

```

Enter number of processes: 4

Process 1

Arrival Time: 0

Burst Time: 4

Type (1=System,2=User): 1

Process 2

Arrival Time: 0

Burst Time: 3

Type (1=System,2=User): 1

Process 3

Arrival Time: 0

Burst Time: 8

Type (1=System,2=User): 2

Process 4

Arrival Time: 10

Burst Time: 5

Type (1=System,2=User): 1

---System Queue---

PID	AT	BT	CT	TAT	WT
1	0	4	4	4	0
2	0	3	7	7	4
4	10	5	15	5	0

---User Queue---

PID	AT	BT	CT	TAT	WT
3	0	8	23	23	15

Average Turnaround Time=9.75

Average Waiting Time=4.75

Naysha Navin Shrivastava

3. Write a C program to simulate Real-Time CPU Scheduling algorithms:

- a) Rate- Monotonic**
- b) Earliest-deadline First**
- c) Proportional scheduling**

Code:

```
#include<stdio.h>

int gcd(int a,int b){return b==0?a:gcd(b,a%b);}

int lcm(int a,int b){return (a*b)/gcd(a,b);}

int main(){
int n,i,j,time=0;

printf("Enter the number of processes:");

scanf("%d",&n);

int burst[n],period[n],rem[n];

printf("Enter the CPU burst times:\n");
for(i=0;i<n;i++)scanf("%d",&burst[i]);

printf("Enter the time periods:\n");
for(i=0;i<n;i++)scanf("%d",&period[i]);

for(i=0;i<n;i++)rem[i]=0;

int hyper=period[0];

for(i=1;i<n;i++)hyper=lcm(hyper,period[i]);

printf("LCM=%d\n\n",hyper);

printf("Rate Monotone Scheduling:\n");

printf("PID\tBurst\tPeriod\n");

for(i=0;i<n;i++)printf("%d\t%d\t%d\n",i+1,burst[i],period[i]);

printf("\nScheduling occurs for %d ms\n\n",hyper);

while(time<hyper){
for(i=0;i<n;i++){
if(time%period[i]==0)rem[i]=burst[i];
}

int idx=-1,minp=9999;

for(i=0;i<n;i++){
```

```

if(rem[i]>0 && period[i]<minp){
minp=period[i];

idx=i;
}

}

if(idx==-1){
printf("%dms onwards: CPU is idle\n",time); }

else{
printf("%dms onwards: Process %d running\n",time,idx+1);
rem[idx]--;
}

time++;
}

return 0;
}

```

Output:

```

C:\Users\BMSCECSE-L4\Desktop X + v
Enter the number of processes:2
Enter the CPU burst times:
20 35
Enter the time periods:
50 100
LCM=100

Rate Monotone Scheduling:
PID    Burst   Period
1       20      50
2       35      100

Scheduling occurs for 100 ms

0ms onwards: Process 1 running
1ms onwards: Process 1 running
2ms onwards: Process 1 running
3ms onwards: Process 1 running
4ms onwards: Process 1 running
5ms onwards: Process 1 running
6ms onwards: Process 1 running
7ms onwards: Process 1 running
8ms onwards: Process 1 running
9ms onwards: Process 1 running
10ms onwards: Process 1 running
11ms onwards: Process 1 running
12ms onwards: Process 1 running
13ms onwards: Process 1 running
14ms onwards: Process 1 running
15ms onwards: Process 1 running

```

```
C:\Users\BMSCECSE-L4\Desktop X + v
71ms onwards: Process 2 running
72ms onwards: Process 2 running
73ms onwards: Process 2 running
74ms onwards: Process 2 running
75ms onwards: CPU is idle
76ms onwards: CPU is idle
77ms onwards: CPU is idle
78ms onwards: CPU is idle
79ms onwards: CPU is idle
80ms onwards: CPU is idle
81ms onwards: CPU is idle
82ms onwards: CPU is idle
83ms onwards: CPU is idle
84ms onwards: CPU is idle
85ms onwards: CPU is idle
86ms onwards: CPU is idle
87ms onwards: CPU is idle
88ms onwards: CPU is idle
89ms onwards: CPU is idle
90ms onwards: CPU is idle
91ms onwards: CPU is idle
92ms onwards: CPU is idle
93ms onwards: CPU is idle
94ms onwards: CPU is idle
95ms onwards: CPU is idle
96ms onwards: CPU is idle
97ms onwards: CPU is idle
98ms onwards: CPU is idle
99ms onwards: CPU is idle
```

b) Earliest-deadline First

Code:

```
#include <stdio.h>

#define MAX 10

int main() {
    int n;
    int burst[MAX], deadline[MAX], period[MAX];
    int remaining[MAX];
    int time = 0;
    printf("Enter number of tasks: ");
    scanf("%d", &n);
    for (int i = 0; i < n; i++) {
        printf("Task %d (burst deadline period): ", i + 1);
        scanf("%d %d %d", &burst[i], &deadline[i],
            &period[i]); remaining[i] = 0; // initially no job released
    }
    printf("\nPID\tBurst\tDeadline\tPeriod\n");
    for (int i = 0; i < n; i++) {
        printf("%d\t%d\t%d\t\t%d\n", i + 1, burst[i], deadline[i],
            period[i]); }
    int limit = 20;
    printf("\nScheduling occurs for %d ms\n\n",
        limit); while (time < limit) {
        for (int i = 0; i < n; i++) {
            if (time % period[i] == 0) {
                remaining[i] = burst[i];
            }
        }
        int min_deadline = 9999;
        int selected = -1;
        for (int i = 0; i < n; i++) {
```

```
if (remaining[i] > 0) {  
    if (deadline[i] < min_deadline) {  
        min_deadline = deadline[i];  
        selected = i;  
    }  
}  
  
if (selected == -1) {  
    printf("%dms : CPU is idle.\n", time);  
} else {  
    printf("%dms : Task %d is running.\n", time, selected +  
1); remaining[selected]--;  
}  
time++;  
}  
return 0;  
}
```

Output:


```
C:\Users\BMSCECSE-14\Desktop X + v
Maysha Navin Shrivastava 1WA24CS184
Enter number of tasks: 2
Task 1 (burst deadline period): 2
4
5
Task 2 (burst deadline period): 3
7
20

PID    Burst    Deadline    Period
1       2         4           5
2       3         7           20

Scheduling occurs for 20 ms

0ms : Task 1 is running.
1ms : Task 1 is running.
2ms : Task 2 is running.
3ms : Task 2 is running.
4ms : Task 2 is running.
5ms : Task 1 is running.
6ms : Task 1 is running.
7ms : CPU is idle.
8ms : CPU is idle.
9ms : CPU is idle.
10ms : Task 1 is running.
11ms : Task 1 is running.
12ms : CPU is idle.
13ms : CPU is idle.
14ms : CPU is idle.
15ms : Task 1 is running.
16ms : Task 1 is running.
17ms : CPU is idle.
18ms : CPU is idle.
19ms : CPU is idle.

Process returned 0 (0x0)   execution time : 37.760 s
Press any key to continue.
```

c) Proportional Scheduling

```
#include <stdio.h>
```

```
#define MAX 10
```

```
typedef struct
{
    int pid;
    int burst;
    int period;
} Task;
```

```
int gcd(int a, int b)
{
    while (b != 0)
    {
        int t = b;
        b = a % b;
        a = t;
    }
    return a;
}
```

```
int lcm(int a, int b)
{

```

```

    return (a * b) / gcd(a, b);
}

int find_lcm(Task t[], int n)
{
    int res, i;

    res = t[0].period;

    for (i = 1; i < n; i++)
        res = lcm(res, t[i].period);

    return res;
}

void sort(Task t[], int n)
{
    int i, j;
    Task temp;

    for (i = 0; i < n - 1; i++)
    {
        for (j = i + 1; j < n; j++)
        {
            if (t[i].period > t[j].period)
            {
                temp = t[i];
                t[i] = t[j];
                t[j] = temp;
            }
        }
    }
}

int main()
{
    int n, i, time;
    int hyper;
    int remaining[MAX];
    int running;
    int prev;
    float util;
    float bound;

    Task t[MAX];

    printf("Enter the number of processes: ");
    scanf("%d", &n);

```

```

printf("Enter the CPU burst times:\n");
for (i = 0; i < n; i++)
{
    scanf("%d", &t[i].burst);
    t[i].pid = i + 1;
}

printf("Enter the time periods:\n");
for (i = 0; i < n; i++)
{
    scanf("%d", &t[i].period);
}

sort(t, n);

hyper = find_lcm(t, n);

printf("LCM=%d\n\n", hyper);

printf("Rate Monotone Scheduling:\n");
printf("PID\tBurst\tPeriod\n");

for (i = 0; i < n; i++)
{
    printf("%d\t%d\t%d\n",
        t[i].pid,
        t[i].burst,
        t[i].period);
}

util = 0.0;

for (i = 0; i < n; i++)
{
    util += (float)t[i].burst / t[i].period;
}

if (n == 1)
    bound = 1.0;
else if (n == 2)
    bound = 0.828427;
else if (n == 3)
    bound = 0.779763;
else if (n == 4)
    bound = 0.756828;
else
    bound = 0.693;

printf("\n%f <= %f => %s\n",

```

```

    util,
    bound,
    (util <= bound) ? "true" : "false");

printf("Scheduling occurs for %d ms\n\n", hyper);

for (i = 0; i < MAX; i++)
    remaining[i] = 0;

prev = -2;

for (time = 0; time < hyper; time++)
{
    for (i = 0; i < n; i++)
    {
        if (time % t[i].period == 0)
        {
            remaining[i] = t[i].burst;
        }
    }

    running = -1;

    for (i = 0; i < n; i++)
    {
        if (remaining[i] > 0)
        {
            running = i;
            break;
        }
    }

    if (running != prev)
    {
        if (running == -1)
            printf("%dms onwards: CPU is idle\n", time);
        else
            printf("%dms onwards: Process %d running\n",
                time,
                t[running].pid);

        prev = running;
    }

    if (running != -1)
        remaining[running]--;
}

return 0;

```

```
}
```

Output

```
Enter the number of Processes: 3

Process 1:
Tickets: 10

Process 2:
Tickets: 20

Process 3:
Tickets: 30

--- Proportional Share Scheduling ---
Enter the Time Period for scheduling: 5
Tickets picked: 18, Winner: P2
Tickets picked: 9, Winner: P1
Tickets picked: 10, Winner: P1
Tickets picked: 51, Winner: P3
Tickets picked: 51, Winner: P3

The Process: P1 gets 16.67% of Processor Time.

The Process: P2 gets 33.33% of Processor Time.

The Process: P3 gets 50.00% of Processor Time.
```

4) Write a C program to simulate:

- a) Producer-Consumer problem using semaphores.
- b) Dining-Philosopher's problem

(a) Producer - Consumer Problem using semaphores

Code:

```
#include <pthread.h>
#include <semaphore.h>
#include <stdio.h>
#include <unistd.h>

#define BUFFER_SIZE 5

int buffer[BUFFER_SIZE];
int in = 0, out = 0;

sem_t empty;
sem_t full;
pthread_mutex_t mutex;

void *producer(void *arg)
```

```

{
    int item, i = 0;

    while (1)
    {
        item = i++;

        sem_wait(&empty);
        pthread_mutex_lock(&mutex);

        buffer[in] = item;
        printf("Produced: %d at buffer[%d]\n", item, in);

        in = (in + 1) % BUFFER_SIZE;

        pthread_mutex_unlock(&mutex);
        sem_post(&full);

        sleep(1);
    }
}

void *consumer(void *arg)
{
    int item;

    while (1)
    {
        sem_wait(&full);
        pthread_mutex_lock(&mutex);

        item = buffer[out];
        printf("Consumed: %d from buffer[%d]\n", item, out);

        out = (out + 1) % BUFFER_SIZE;

        pthread_mutex_unlock(&mutex);
        sem_post(&empty);

        sleep(2);
    }
}

int main()
{
    pthread_t prod, cons;

```

```
sem_init(&empty, 0, BUFFER_SIZE);
sem_init(&full, 0, 0);

pthread_mutex_init(&mutex, NULL);

pthread_create(&prod, NULL, producer, NULL);
pthread_create(&cons, NULL, consumer, NULL);

pthread_join(prod, NULL);
pthread_join(cons, NULL);

sem_destroy(&empty);
sem_destroy(&full);

pthread_mutex_destroy(&mutex);

return 0;
}
```

```
Consumed: 0 from buffer[0]
Produced: 1 at buffer[1]
Consumed: 1 from buffer[1]
Produced: 2 at buffer[2]
Produced: 3 at buffer[3]
Consumed: 2 from buffer[2]
Produced: 4 at buffer[4]
Produced: 5 at buffer[0]
Consumed: 3 from buffer[3]
Produced: 6 at buffer[1]
Produced: 7 at buffer[2]
Consumed: 4 from buffer[4]
Produced: 8 at buffer[3]
Produced: 9 at buffer[4]
Consumed: 5 from buffer[0]
Produced: 10 at buffer[0]
Consumed: 6 from buffer[1]
Produced: 11 at buffer[1]
Consumed: 7 from buffer[2]
Produced: 12 at buffer[2]
Consumed: 8 from buffer[3]
Produced: 13 at buffer[3]
Consumed: 9 from buffer[4]
Produced: 14 at buffer[4]
```

(B) Dining-Philosopher's problem


```

#include <pthread.h>
#include <stdio.h>
#include <unistd.h>

#define N 5

pthread_mutex_t forks[N];
pthread_t philosophers[N];

void *philosopher(void *num)
{
    int id = *(int *)num;
    int left = id;
    int right = (id + 1) % N;

    while (1)
    {
        printf("Philosopher %d is thinking.\n", id);
        sleep(1);

        if (id % 2 == 0)
        {
            pthread_mutex_lock(&forks[left]);
            printf("Philosopher %d picked up left fork %d.\n", id, left);

            pthread_mutex_lock(&forks[right]);
            printf("Philosopher %d picked up right fork %d.\n", id, right);
        }
        else
        {
            pthread_mutex_lock(&forks[right]);
            printf("Philosopher %d picked up right fork %d.\n", id, right);

            pthread_mutex_lock(&forks[left]);
            printf("Philosopher %d picked up left fork %d.\n", id, left);
        }

        printf("Philosopher %d is eating.\n", id);
        sleep(2);

        pthread_mutex_unlock(&forks[left]);
        pthread_mutex_unlock(&forks[right]);

        printf("Philosopher %d put down forks %d and %d.\n",
            id, left, right);
    }

    return NULL;
}

```

```
int main()
{

    int i;
    int ids[N];

    for (i = 0; i < N; i++)
    {
        pthread_mutex_init(&forks[i], NULL);
        ids[i] = i;
    }

    for (i = 0; i < N; i++)
    {
        pthread_create(&philosophers[i], NULL,
                      philosopher, &ids[i]);
    }

    for (i = 0; i < N; i++)
    {
        pthread_join(philosophers[i], NULL);
    }

    for (i = 0; i < N; i++)
    {
        pthread_mutex_destroy(&forks[i]);
    }

    return 0;
}
```

Philosopher 0 is thinking.
Philosopher 1 is thinking.
Philosopher 2 is thinking.
Philosopher 3 is thinking.
Philosopher 4 is thinking.
Philosopher 0 picked up left fork 0.
Philosopher 0 picked up right fork 1.
Philosopher 0 is eating.
Philosopher 1 picked up right fork 2.
Philosopher 3 picked up right fork 4.
Philosopher 3 picked up left fork 3.
Philosopher 3 is eating.
Philosopher 0 put down forks 0 and 1.
Philosopher 0 is thinking.
Philosopher 1 picked up left fork 1.
Philosopher 1 is eating.
Philosopher 3 put down forks 3 and 4.
Philosopher 3 is thinking.
Philosopher 4 picked up left fork 4.
Philosopher 4 picked up right fork 0.
Philosopher 4 is eating.
Philosopher 1 put down forks 1 and 2.
Philosopher 1 is thinking.
Philosopher 2 picked up left fork 2.
Philosopher 2 picked up right fork 3.

5)Write a C program to simulate:

a) Bankers' algorithm for the purpose of deadlock avoidance.

b) Deadlock Detection

(A) Bankers' algorithm for the purpose of deadlock avoidance.

Code:

```
#include <stdio.h>

int main()
{
    int p, r, i, j, k;

    printf("Naysha Navin Shrivastava \n");
    printf("1WA24CS184 \n");

    printf("Enter number of processes: ");
    scanf("%d", &p);

    printf("Enter number of resource types: ");
    scanf("%d", &r);

    int allocation[p][r];
    int request[p][r];
    int available[r];

    printf("\nEnter Allocation Matrix:\n");
    for(i = 0; i < p; i++)
    {
        for(j = 0; j < r; j++)
        {
            scanf("%d", &allocation[i][j]);
        }
    }

    printf("\nEnter Request Matrix:\n");
    for(i = 0; i < p; i++)
    {
        for(j = 0; j < r; j++)
        {
            scanf("%d", &request[i][j]);
        }
    }

    printf("\nEnter Available Resources:\n");
    for(i = 0; i < r; i++)
    {
        scanf("%d", &available[i]);
    }
}
```

```

int finish[p];
int work[r];
int sequence[p];
int count = 0;

for(i = 0; i < r; i++)
{
    work[i] = available[i];
}

for(i = 0; i < p; i++)
{
    int flag = 0;

    for(j = 0; j < r; j++)
    {
        if(allocation[i][j] != 0)
        {
            flag = 1;
            break;
        }
    }

    if(flag == 0)
        finish[i] = 1;
    else
        finish[i] = 0;
}

int found;

do
{
    found = 0;

    for(i = 0; i < p; i++)
    {
        if(finish[i] == 0)
        {
            for(j = 0; j < r; j++)
            {
                if(request[i][j] > work[j])
                {
                    break;
                }
            }

            if(j == r)

```

```

        {
            for(k = 0; k < r; k++)
            {
                work[k] += allocation[i][k];
            }

            sequence[count++] = i;
            finish[i] = 1;
            found = 1;
        }
    }
}

} while(found);

int deadlock = 0;

printf("\nExecution Sequence:\n");

for(i = 0; i < count; i++)
{
    printf("P%d", sequence[i]);

    if(i != count - 1)
    {
        printf(" -> ");
    }
}

printf("\n");

printf("\nProcesses in Deadlock:\n");

for(i = 0; i < p; i++)
{
    if(finish[i] == 0)
    {
        printf("P%d ", i);
        deadlock = 1;
    }
}

if(deadlock == 0)
{
    printf("No Deadlock Detected");
}

printf("\n");

```

```
    return 0;
}
```

```
Naysha Navin Shrivastava
USN:1WA24CS184
Enter number of processes: 5
Enter number of resource types: 3

Enter Allocation Matrix:
0 1 0
2 0 0
3 0 2

2 1 1
0 0 2

Enter Maximum Matrix:
7 5 3
3 2 2
9 0 2
2 2 2
4 3 3

Enter Available Resources:
3 3 2

Need Matrix:
7 4 3
1 2 2
6 0 0
0 1 1
4 3 1

System is in SAFE State
Safe Sequence:
P1 -> P3 -> P4 -> P0 -> P2
```

(B) Deadlock Detection

Code:

```
#include <stdio.h>
```

```
int main()
{
    int p, r, i, j, k;

    printf("Naysha Navin Shrivastava \n");
    printf("1WA24CS184 \n");

    printf("Enter number of processes: ");
    scanf("%d", &p);

    printf("Enter number of resource types: ");
```

```

scanf("%d", &r);

int allocation[p][r];
int request[p][r];
int available[r];

printf("\nEnter Allocation Matrix:\n");
for(i = 0; i < p; i++)
{
    for(j = 0; j < r; j++)
    {
        scanf("%d", &allocation[i][j]);
    }
}

printf("\nEnter Request Matrix:\n");
for(i = 0; i < p; i++)
{
    for(j = 0; j < r; j++)
    {
        scanf("%d", &request[i][j]);
    }
}

printf("\nEnter Available Resources:\n");
for(i = 0; i < r; i++)
{
    scanf("%d", &available[i]);
}

int finish[p];
int work[r];
int sequence[p];
int count = 0;

for(i = 0; i < r; i++)
{
    work[i] = available[i];
}

for(i = 0; i < p; i++)
{
    int flag = 0;

    for(j = 0; j < r; j++)
    {
        if(allocation[i][j] != 0)
        {
            flag = 1;

```



```

        break;
    }
}

if(flag == 0)
    finish[i] = 1;
else
    finish[i] = 0;
}

int found;

do
{
    found = 0;

    for(i = 0; i < p; i++)
    {
        if(finish[i] == 0)
        {
            for(j = 0; j < r; j++)
            {
                if(request[i][j] > work[j])
                {
                    break;
                }
            }

            if(j == r)
            {
                for(k = 0; k < r; k++)
                {
                    work[k] += allocation[i][k];
                }

                sequence[count++] = i;
                finish[i] = 1;
                found = 1;
            }
        }
    }
} while(found);

int deadlock = 0;

printf("\nExecution Sequence:\n");

for(i = 0; i < count; i++)

```

```
{
    printf("P%d", sequence[i]);

    if(i != count - 1)
    {
        printf(" -> ");
    }
}

printf("\n");

printf("\nProcesses in Deadlock:\n");

for(i = 0; i < p; i++)
{
    if(finish[i] == 0)
    {
        printf("P%d ", i);
        deadlock = 1;
    }
}

if(deadlock == 0)
{
    printf("No Deadlock Detected");
}

printf("\n");

return 0;
}
```

```

Naysha Navin Shrivastava
1WA24CS184
Enter number of processes: 5
Enter number of resource types: 3

Enter Allocation Matrix:
0 1 0
2 0 0
3 0 3
2 1 1
0 0 2

Enter Request Matrix:
0 0 0
2 0 2
0 0 0
1 0 0
0 0 2

Enter Available Resources:
0 0 0

Execution Sequence:
P0 -> P2 -> P3 -> P4 -> P1

Processes in Deadlock:
No Deadlock Detected

=== Code Execution Successful ===

```

6. Write a C program to simulate the following contiguous memory allocation techniques.

- a) Worst-fit**
- b) Best-fit**
- c) First-fit**

(A)Worst-Fit

```
#include <stdio.h>
```

```
void firstFit(int blockSize[], int m, int processSize[], int n)
```

```
{
    int allocation[n];

    for(int i = 0; i < n; i++)
        allocation[i] = -1;

    for(int i = 0; i < n; i++)
    {
        for(int j = 0; j < m; j++)
        {
            if(blockSize[j] >= processSize[i])
            {
                allocation[i] = j;
                blockSize[j] -= processSize[i];
                break;
            }
        }
    }
}
```

```

    }
}

printf("\nFIRST FIT Allocation\n");
printf("Process No.\tProcess Size\tBlock No.\n");

for(int i = 0; i < n; i++)
{
    printf("%d\t%d\t", i + 1, processSize[i]);

    if(allocation[i] != -1)
        printf("%d\n", allocation[i] + 1);
    else
        printf("Not Allocated\n");
}

}

void bestFit(int blockSize[], int m, int processSize[], int n)
{
    int allocation[n];

    for(int i = 0; i < n; i++)
        allocation[i] = -1;

    for(int i = 0; i < n; i++)
    {
        int bestIdx = -1;

        for(int j = 0; j < m; j++)
        {
            if(blockSize[j] >= processSize[i])
            {
                if(bestIdx == -1 || blockSize[j] < blockSize[bestIdx])
                    bestIdx = j;
            }
        }

        if(bestIdx != -1)
        {
            allocation[i] = bestIdx;
            blockSize[bestIdx] -= processSize[i];
        }
    }

    printf("\nBEST FIT Allocation\n");
    printf("Process No.\tProcess Size\tBlock No.\n");

    for(int i = 0; i < n; i++)

```

```

    {
        printf("%d\t\t%d\t\t", i + 1, processSize[i]);

        if(allocation[i] != -1)
            printf("%d\n", allocation[i] + 1);
        else
            printf("Not Allocated\n");
    }
}

void worstFit(int blockSize[], int m, int processSize[], int n)
{
    int allocation[n];

    for(int i = 0; i < n; i++)
        allocation[i] = -1;

    for(int i = 0; i < n; i++)
    {
        int worstIdx = -1;

        for(int j = 0; j < m; j++)
        {
            if(blockSize[j] >= processSize[i])
            {
                if(worstIdx == -1 || blockSize[j] > blockSize[worstIdx])
                    worstIdx = j;
            }
        }

        if(worstIdx != -1)
        {
            allocation[i] = worstIdx;
            blockSize[worstIdx] -= processSize[i];
        }
    }

    printf("\nWORST FIT Allocation\n");
    printf("Process No.\tProcess Size\tBlock No.\n");

    for(int i = 0; i < n; i++)
    {
        printf("%d\t\t%d\t\t", i + 1, processSize[i]);

        if(allocation[i] != -1)
            printf("%d\n", allocation[i] + 1);
        else
            printf("Not Allocated\n");
    }
}

```

```

}

int main()
{
    int m, n;

    printf("Naysha Navin Shrivastava\n");
    printf("1WA24CS184\n");

    printf("\nEnter number of memory blocks: ");
    scanf("%d", &m);

    int blockSize[m];
    int block1[m], block2[m], block3[m];

    printf("Enter sizes of memory blocks:\n");
    for(int i = 0; i < m; i++)
    {
        scanf("%d", &blockSize[i]);
        block1[i] = block2[i] = block3[i] = blockSize[i];
    }

    printf("\nEnter number of processes: ");
    scanf("%d", &n);

    int processSize[n];

    printf("Enter sizes of processes:\n");
    for(int i = 0; i < n; i++)
        scanf("%d", &processSize[i]);

    firstFit(block1, m, processSize, n);
    bestFit(block2, m, processSize, n);
    worstFit(block3, m, processSize, n);

    return 0;
}

```

```
Enter number of memory blocks: 5
```

```
Enter sizes of memory blocks:
```

```
400
```

```
700
```

```
200
```

```
300
```

```
600
```

```
Enter number of processes: 4
```

```
Enter sizes of processes:
```

```
212
```

```
512
```

```
312
```

```
526
```

```
First Fit Allocation:
```

```
Process 1 -> Block 1
```

```
Process 2 -> Block 2
```

```
Process 3 -> Block 5
```

```
Process 4 -> Not Allocated
```

```
Best Fit Allocation:
```

```
Process 1 -> Block 4
```

```
Process 2 -> Block 5
```

```
Process 3 -> Block 1
```

```
Process 4 -> Block 2
```

```
Worst Fit Allocation:
```

```
Process 1 -> Block 2
```

```
Process 2 -> Block 5
```

```
Process 3 -> Block 2
```

```
Process 4 -> Not Allocated
```

```
=== Code Execution Successful ===
```

7. Write a C program to simulate page replacement algorithms.

a) FIFO

b) LRU

c) Optimal

```
#include <stdio.h>
```

```
void printFrames(int frames[], int f)
```

```
{
```

```
    for(int i = 0; i < f; i++)
```

```

    {
        if(frames[i] == -1)
            printf("-\t");
        else
            printf("%d\t", frames[i]);
    }
}

void FIFO(int pages[], int n, int f)
{
    int frames[20];
    int pointer = 0, hits = 0, misses = 0;

    for(int i = 0; i < f; i++)
        frames[i] = -1;

    printf("\n\n===== FIFO =====\n");
    printf("Page\tFrames\t\t\tStatus\n");

    for(int i = 0; i < n; i++)
    {
        int found = 0;

        for(int j = 0; j < f; j++)
        {
            if(frames[j] == pages[i])
            {
                found = 1;
                hits++;
                break;
            }
        }

        if(!found)
        {
            frames[pointer] = pages[i];
            pointer = (pointer + 1) % f;
            misses++;
        }

        printf("%d\t", pages[i]);
        printFrames(frames, f);
        printf("%s\n", found ? "Hit" : "Miss");
    }

    printf("\nSummary:\n");
    printf("Total Accesses = %d\n", n);
    printf("Total Hits = %d\n", hits);
    printf("Total Misses = %d\n", misses);
}

```



```

printf("Hit Ratio = %.2f\n", (float)hits/n);
printf("Miss Ratio = %.2f\n", (float)misses/n);
}

```

```

void LRU(int pages[], int n, int f)
{
    int frames[20], time[20];
    int hits = 0, misses = 0, counter = 0;

```

```

    for(int i = 0; i < f; i++)
    {
        frames[i] = -1;
        time[i] = 0;
    }

```

```

    printf("\n\n===== LRU =====\n");
    printf("Page\tFrames\t\t\tStatus\n");

```

```

    for(int i = 0; i < n; i++)
    {
        int found = 0;

        for(int j = 0; j < f; j++)
        {
            if(frames[j] == pages[i])
            {
                found = 1;
                hits++;
                counter++;
                time[j] = counter;
                break;
            }
        }
    }

```

```

    if(!found)
    {
        int pos = 0;

```

```

        for(int j = 1; j < f; j++)
        {
            if(frames[j] == -1)
            {
                pos = j;
                break;
            }

```

```

            if(time[j] < time[pos])
                pos = j;
        }
    }

```

```

        frames[pos] = pages[i];
        counter++;
        time[pos] = counter;
        misses++;
    }

    printf("%d\t", pages[i]);
    printFrames(frames, f);
    printf("%s\n", found ? "Hit" : "Miss");
}

printf("\nSummary:\n");
printf("Total Accesses = %d\n", n);
printf("Total Hits = %d\n", hits);
printf("Total Misses = %d\n", misses);
printf("Hit Ratio = %.2f\n", (float)hits/n);
printf("Miss Ratio = %.2f\n", (float)misses/n);
}

void OPTIMAL(int pages[], int n, int f)
{
    int frames[20];
    int hits = 0, misses = 0;

    for(int i = 0; i < f; i++)
        frames[i] = -1;

    printf("\n\n===== OPTIMAL =====\n");
    printf("Page\tFrames\t\t\tStatus\n");

    for(int i = 0; i < n; i++)
    {
        int found = 0;

        for(int j = 0; j < f; j++)
        {
            if(frames[j] == pages[i])
            {
                found = 1;
                hits++;
                break;
            }
        }

        if(!found)
        {
            int empty = -1;

```

```

for(int j = 0; j < f; j++)
{
    if(frames[j] == -1)
    {
        empty = j;
        break;
    }
}

if(empty != -1)
{
    frames[empty] = pages[i];
}
else
{
    int farthest = -1;
    int pos = -1;

    for(int j = 0; j < f; j++)
    {
        int k;

        for(k = i + 1; k < n; k++)
        {
            if(frames[j] == pages[k])
                break;
        }

        if(k == n)
        {
            pos = j;
            break;
        }

        if(k > farthest)
        {
            farthest = k;
            pos = j;
        }
    }

    frames[pos] = pages[i];
}

misses++;
}

printf("%d\t", pages[i]);
printFrames(frames, f);

```

```

        printf("%s\n", found ? "Hit" : "Miss");
    }

    printf("\nSummary:\n");
    printf("Total Accesses = %d\n", n);
    printf("Total Hits = %d\n", hits);
    printf("Total Misses = %d\n", misses);
    printf("Hit Ratio = %.2f\n", (float)hits/n);
    printf("Miss Ratio = %.2f\n", (float)misses/n);
}

int main()
{
    int n, f;

    printf("Naysha Navin Shrivastava\n");
    printf("1WA24CS184\n");

    printf("\nEnter number of pages: ");
    scanf("%d", &n);

    int pages[n];

    printf("Enter page reference string:\n");
    for(int i = 0; i < n; i++)
        scanf("%d", &pages[i]);

    printf("Enter number of frames: ");
    scanf("%d", &f);

    FIFO(pages, n, f);
    LRU(pages, n, f);
    OPTIMAL(pages, n, f);

    return 0;
}

```

```

Enter number of pages: 12
Enter the reference string: 7
0 1 2 0 3 0 4 2 3 0 3
Enter number of frames: 3

```

```

--- FIFO ---

```

Page	Frames	
7	7 - -	*MISS*
0	7 0 -	*MISS*
1	7 0 1	*MISS*
2	2 0 1	*MISS*
0	2 0 1	*HIT*
3	2 3 1	*MISS*
0	2 3 0	*MISS*
4	4 3 0	*MISS*
2	4 2 0	*MISS*
3	4 2 3	*MISS*
0	0 2 3	*MISS*
3	0 2 3	*HIT*

```

[FIFO Summary]

```

```

Total Accesses : 12
Total Hits      : 2
Total Misses    : 10
Hit/Miss Ratio : 20.00%

```

```

--- LRU ---

```

Page	Frames	
7	7 - -	*MISS*
0	7 0 -	*MISS*
1	7 0 1	*MISS*
2	2 0 1	*MISS*
0	2 0 1	*HIT*
3	2 0 3	*MISS*
0	2 0 3	*HIT*
4	4 0 3	*MISS*
2	4 0 2	*MISS*
3	4 3 2	*MISS*
0	0 3 2	*MISS*
3	0 3 2	*HIT*

```

[LRU Summary]

```

```

Total Accesses : 12
Total Hits      : 3
Total Misses    : 9
Hit/Miss Ratio : 33.33%

```

```

Total Accesses : 12
Total Hits      : 5
Total Misses    : 7
Hit/Miss Ratio : 38.38%

```

```

--- Optimal ---

```

Page	Frames	
7	7 - -	*MISS*
0	7 0 -	*MISS*
1	7 0 1	*MISS*
2	2 0 1	*MISS*
0	2 0 1	*HIT*
3	2 0 3	*MISS*
0	2 0 3	*HIT*
4	2 4 3	*MISS*
1	2 4 3	*HIT*
3	2 4 3	*HIT*
0	0 4 3	*MISS*
3	0 4 3	*HIT*

```

[Optimal Summary]

```

```

Total Accesses : 12
Total Hits      : 5
Total Misses    : 7
Hit/Miss Ratio : 71.43%

```

```

--- Code Execution Successful ---

```

